JavaScript is disabled on your browser.

Java SE 27 & JDK 27

Class ZonedDateTime

java.lang.Object
    java.time.ZonedDateTime

All Implemented Interfaces:

`Serializable`, `Comparable<ChronoZonedDateTime<?>>`, `ChronoZonedDateTime<LocalDate>`, `Temporal`, `TemporalAccessor`

```
public final class ZonedDateTime
extends Object
implements Temporal, ChronoZonedDateTime<LocalDate>, Serializable
```

A date-time with a time-zone in the ISO-8601 calendar system, such as `2007-12-03T10:15:30+01:00 Europe/Paris`.

`ZonedDateTime` is an immutable representation of a date-time with a time-zone. This class stores all date and time fields, to a precision of nanoseconds, and a time-zone, with a zone offset used to handle ambiguous local date-times. For example, the value "2nd October 2007 at 13:45.30.123456789 +02:00 in the Europe/Paris time-zone" can be stored in a `ZonedDateTime`.

This class handles conversion from the local time-line of `LocalDateTime` to the instant time-line of `Instant`. The difference between the two time-lines is the offset from UTC/Greenwich, represented by a `ZoneOffset`.

Converting between the two time-lines involves calculating the offset using the `rules` accessed from the `ZoneId`. Obtaining the offset for an instant is simple, as there is exactly one valid offset for each instant. By contrast, obtaining the offset for a local date-time is not straightforward. There are three cases:

- Normal, with one valid offset. For the vast majority of the year, the normal case applies, where there is a single valid offset for the local date-time.
- Gap, with zero valid offsets. This is when clocks jump forward typically due to the spring daylight savings change from "winter" to "summer". In a gap there are local date-time values with no valid offset.
- Overlap, with two valid offsets. This is when clocks are set back typically due to the autumn daylight savings change from "summer" to "winter". In an overlap there are local date-time values with two valid offsets.

Any method that converts directly or implicitly from a local date-time to an instant by obtaining the offset has the potential to be complicated.

For Gaps, the general strategy is that if the local date-time falls in the middle of a Gap, then the resulting zoned date-time will have a local date-time shifted forwards by the length of the Gap, resulting in a date-time in the later offset, typically "summer" time.

For Overlaps, the general strategy is that if the local date-time falls in the middle of an Overlap, then the previous offset will be retained. If there is no previous offset, or the previous offset is invalid, then the earlier offset is used, typically "summer" time. Two additional methods, `withEarlierOffsetAtOverlap()` and `withLaterOffsetAtOverlap()`, help manage the case of an overlap.

In terms of design, this class should be viewed primarily as the combination of a `LocalDateTime` and a `ZoneId`. The `ZoneOffset` is a vital, but secondary, piece of information, used to ensure that the class represents an instant, especially during a daylight savings overlap.

This is a value-based class; programmers should treat instances that are equal as interchangeable and should not use instances for synchronization, or unpredictable behavior may occur. For example, in a future release, synchronization may fail. The `equals` method should be used for comparisons.

Implementation Requirements:

A `ZonedDateTime` holds state equivalent to three separate objects, a `LocalDateTime`, a `ZoneId` and the resolved `ZoneOffset`. The offset and local date-time are used to define an instant when necessary. The zone ID is used to obtain the rules for how and when the offset changes. The offset cannot be freely set, as the zone controls which offsets are valid.

This class is immutable and thread-safe.

Since:

1.8

See Also:

Serialized Form

Method Summary

All Methods	Static Methods	
Modifier and Type	**Method**	**Description**
`boolean`	`equals(Object obj)`	Checks if this date-time is equal to another date-time.
`String`	`format (DateTimeFormatter forma`	Formats this date-time using the specified formatter.
`static ZonedDateTime`	`from (TemporalAccessor tempor`	

		Obtains an instance of `ZonedDateTime` from a temporal object.
`int`	**`get`**`(TemporalField field)`	Gets the value of the specified field from this date-time as an `int`.
`int`	**`getDayOfMonth`**`()`	Gets the day-of-month field.
`DayOfWeek`	**`getDayOfWeek`**`()`	Gets the day-of-week field, which is an enum `DayOfWeek`.
`int`	**`getDayOfYear`**`()`	Gets the day-of-year field.
`int`	**`getHour`**`()`	Gets the hour-of-day field.
`long`	**`getLong`**`(TemporalField field)`	Gets the value of the specified field from this date-time as a `long`.
`int`	**`getMinute`**`()`	Gets the minute-of-hour field.
`Month`	**`getMonth`**`()`	Gets the month-of-year field using the `Month` enum.
`int`	**`getMonthValue`**`()`	Gets the month-of-year field from 1 to 12.
`int`	**`getNano`**`()`	Gets the nano-of-second field.
`ZoneOffset`	**`getOffset`**`()`	Gets the zone offset, such as '+01:00'.
`int`	**`getSecond`**`()`	Gets the second-of-minute field.
`int`	**`getYear`**`()`	Gets the year field.
`ZoneId`	**`getZone`**`()`	Gets the time-zone, such as 'Europe/Paris'.

int	**hashCode**()	A hash code for this date-time.
boolean	**isSupported**(TemporalField field)	Checks if the specified field is supported.
boolean	**isSupported**(TemporalUnit unit)	Checks if the specified unit is supported.
ZonedDateTime	**minus**(long amountToSubtract, TemporalUnit unit)	Returns a copy of this date-time with the specified amount subtracted.
ZonedDateTime	**minus**(TemporalAmount amountTo	Returns a copy of this date-time with the specified amount subtracted.
ZonedDateTime	**minusDays**(long days)	Returns a copy of this ZonedDateTime with the specified number of days subtracted.
ZonedDateTime	**minusHours**(long hours)	Returns a copy of this ZonedDateTime with the specified number of hours subtracted.
ZonedDateTime	**minusMinutes**(long minutes)	Returns a copy of this ZonedDateTime with the specified number of minutes subtracted.
ZonedDateTime	**minusMonths**(long months)	Returns a copy of this ZonedDateTime with the specified number of months subtracted.
ZonedDateTime	**minusNanos**(long nanos)	Returns a copy of this ZonedDateTime with the specified number of nanoseconds subtracted.

ZonedDateTime	**minusSeconds**(long seconds)	Returns a copy of this ZonedDateTime with the specified number of seconds subtracted.
ZonedDateTime	**minusWeeks**(long weeks)	Returns a copy of this ZonedDateTime with the specified number of weeks subtracted.
ZonedDateTime	**minusYears**(long years)	Returns a copy of this ZonedDateTime with the specified number of years subtracted.
static ZonedDateTime	**now**()	Obtains the current date-time from the system clock in the default time-zone.
static ZonedDateTime	**now**(Clock clock)	Obtains the current date-time from the specified clock.
static ZonedDateTime	**now**(ZoneId zone)	Obtains the current date-time from the system clock in the specified time-zone.
static ZonedDateTime	**of**(int year, int month, int dayOfMonth, int hour, int minute, int second, int nanoOfSecond, ZoneId zone)	Obtains an instance of ZonedDateTime from a year, month, day, hour, minute, second, nanosecond and time-zone.
static ZonedDateTime	**of**(LocalDate date, LocalTime time, ZoneId zone)	Obtains an instance of ZonedDateTime from a local date and time.
static ZonedDateTime	**of**(LocalDateTime localDate ZoneId zone)	Obtains an instance of ZonedDateTime from a local date-time.
static ZonedDateTime		

	ofInstant(Instant instant, ZoneId zone)	Obtains an instance of ZonedDateTime from an Instant.
static ZonedDateTime	**ofInstant**(LocalDateTime localDate ZoneOffset offset, ZoneId zone)	Obtains an instance of ZonedDateTime from the instant formed by combining the local date-time and offset.
static ZonedDateTime	**ofLocal**(LocalDateTime localDate ZoneId zone, ZoneOffset preferredOffs	Obtains an instance of ZonedDateTime from a local date-time using the preferred offset if possible.
static ZonedDateTime	**ofStrict**(LocalDateTime localDate ZoneOffset offset, ZoneId zone)	Obtains an instance of ZonedDateTime strictly validating the combination of local date-time, offset and zone ID.
static ZonedDateTime	**parse**(CharSequence text)	Obtains an instance of ZonedDateTime from a text string such as 2007-12-03T10:15:30+01:00[Europe/Paris].
static ZonedDateTime	**parse**(CharSequence text, DateTimeFormatter format	Obtains an instance of ZonedDateTime from a text string using a specific formatter.
ZonedDateTime	**plus**(long amountToAdd, TemporalUnit unit)	Returns a copy of this date-time with the specified amount added.
ZonedDateTime	**plus**(TemporalAmount amountTo	Returns a copy of this date-time with the specified amount added.
ZonedDateTime	**plusDays**(long days)	Returns a copy of this ZonedDateTime with the specified number of days added.

ZonedDateTime	**plusHours**(long hours)	Returns a copy of this ZonedDateTime with the specified number of hours added.
ZonedDateTime	**plusMinutes**(long minutes)	Returns a copy of this ZonedDateTime with the specified number of minutes added.
ZonedDateTime	**plusMonths**(long months)	Returns a copy of this ZonedDateTime with the specified number of months added.
ZonedDateTime	**plusNanos**(long nanos)	Returns a copy of this ZonedDateTime with the specified number of nanoseconds added.
ZonedDateTime	**plusSeconds**(long seconds)	Returns a copy of this ZonedDateTime with the specified number of seconds added.
ZonedDateTime	**plusWeeks**(long weeks)	Returns a copy of this ZonedDateTime with the specified number of weeks added.
ZonedDateTime	**plusYears**(long years)	Returns a copy of this ZonedDateTime with the specified number of years added.
<R> R	**query**(TemporalQuery<R> query)	Queries this date-time using the specified query.
ValueRange	**range**(TemporalField field)	Gets the range of valid values for the specified field.
LocalDate	**toLocalDate**()	Gets the LocalDate part of this date-time.
LocalDateTime	**toLocalDateTime**()	

		Gets the LocalDateTime part of this date-time.
LocalTime	**toLocalTime**()	Gets the LocalTime part of this date-time.
OffsetDateTime	**toOffsetDateTime**()	Converts this date-time to an OffsetDateTime.
String	**toString**()	Outputs this date-time as a String, such as 2007-12-03T10:15:30+01:00[Europe/Paris].
ZonedDateTime	**truncatedTo**(TemporalUnit unit)	Returns a copy of this ZonedDateTime with the time truncated.
long	**until**(Temporal endExclusive, TemporalUnit unit)	Calculates the amount of time until another date-time in terms of the specified unit.
ZonedDateTime	**with**(TemporalAdjuster adjust	Returns an adjusted copy of this date-time.
ZonedDateTime	**with**(TemporalField field, long newValue)	Returns a copy of this date-time with the specified field set to a new value.
ZonedDateTime	**withDayOfMonth**(int dayOfMonth)	Returns a copy of this ZonedDateTime with the day-of-month altered.
ZonedDateTime	**withDayOfYear**(int dayOfYear)	Returns a copy of this ZonedDateTime with the day-of-year altered.
ZonedDateTime	**withEarlierOffsetAtOverl**	Returns a copy of this date-time changing the zone offset to the earlier of the two valid offsets at a local time-line overlap.
ZonedDateTime	**withFixedOffsetZone**()	

		Returns a copy of this date-time with the zone ID set to the offset.
ZonedDateTime	**withHour**(int hour)	Returns a copy of this ZonedDateTime with the hour-of-day altered.
ZonedDateTime	**withLaterOffsetAtOverlap**	Returns a copy of this date-time changing the zone offset to the later of the two valid offsets at a local time-line overlap.
ZonedDateTime	**withMinute**(int minute)	Returns a copy of this ZonedDateTime with the minute-of-hour altered.
ZonedDateTime	**withMonth**(int month)	Returns a copy of this ZonedDateTime with the month-of-year altered.
ZonedDateTime	**withNano**(int nanoOfSecond)	Returns a copy of this ZonedDateTime with the nano-of-second altered.
ZonedDateTime	**withSecond**(int second)	Returns a copy of this ZonedDateTime with the second-of-minute altered.
ZonedDateTime	**withYear**(int year)	Returns a copy of this ZonedDateTime with the year altered.
ZonedDateTime	**withZoneSameInstant**(ZoneId zone)	Returns a copy of this date-time with a different time-zone, retaining the instant.
ZonedDateTime	**withZoneSameLocal**(ZoneId zone)	Returns a copy of this date-time with a different time-zone, retaining the local date-time if possible.

Methods declared in class Object

`clone`, `finalize`, `getClass`, `notify`, `notifyAll`, `wait`, `wait`, `wait`

Methods declared in interface ChronoZonedDateTime

`compareTo`, `getChronology`, `isAfter`, `isBefore`, `isEqual`, `toEpochSecond`, `toInstant`

Method Details

now

```
public static ZonedDateTime now()
```

Obtains the current date-time from the system clock in the default time-zone.

This will query the `system clock` in the default time-zone to obtain the current date-time. The zone and offset will be set based on the time-zone in the clock.

Using this method will prevent the ability to use an alternate clock for testing because the clock is hard-coded.

Returns:

the current date-time using the system clock, not null

now

```
public static ZonedDateTime now(ZoneId zone)
```

Obtains the current date-time from the system clock in the specified time-zone.

This will query the `system clock` to obtain the current date-time. Specifying the time-zone avoids dependence on the default time-zone. The offset will be calculated from the specified time-zone.

Using this method will prevent the ability to use an alternate clock for testing because the clock is hard-coded.

Parameters:

`zone` - the zone ID to use, not null

Returns:

the current date-time using the system clock, not null

now

```
public static ZonedDateTime now(Clock clock)
```

Obtains the current date-time from the specified clock.

This will query the specified clock to obtain the current date-time. The zone and offset will be set based on the time-zone in the clock.

Using this method allows the use of an alternate clock for testing. The alternate clock may be introduced using `dependency injection`.

Parameters:

`clock` - the clock to use, not null

Returns:

the current date-time, not null

of

```
public static ZonedDateTime of(LocalDate date,
                               LocalTime time,
                               ZoneId zone)
```

Obtains an instance of `ZonedDateTime` from a local date and time.

This creates a zoned date-time matching the input local date and time as closely as possible. Time-zone rules, such as daylight savings, mean that not every local date-time is valid for the specified zone, thus the local date-time may be adjusted.

The local date and time are first combined to form a local date-time. The local date-time is then resolved to a single instant on the time-line. This is achieved by finding a valid offset from UTC/Greenwich for the local date-time as defined by the `rules` of the zone ID.

In most cases, there is only one valid offset for a local date-time. In the case of an overlap, when clocks are set back, there are two valid offsets. This method uses the earlier offset typically corresponding to "summer".

In the case of a gap, when clocks jump forward, there is no valid offset. Instead, the local date-time is adjusted to be later by the length of the gap. For a typical one hour daylight savings change, the local date-time will be moved one hour later into the offset typically corresponding to "summer".

Parameters:

`date` - the local date, not null

`time` - the local time, not null

zone - the time-zone, not null

Returns:

the zoned date-time, not null

of

```
public static ZonedDateTime of(LocalDateTime localDateTime,
                               ZoneId zone)
```

Obtains an instance of ZonedDateTime from a local date-time.

This creates a zoned date-time matching the input local date-time as closely as possible. Time-zone rules, such as daylight savings, mean that not every local date-time is valid for the specified zone, thus the local date-time may be adjusted.

The local date-time is resolved to a single instant on the time-line. This is achieved by finding a valid offset from UTC/Greenwich for the local date-time as defined by the rules of the zone ID.

In most cases, there is only one valid offset for a local date-time. In the case of an overlap, when clocks are set back, there are two valid offsets. This method uses the earlier offset typically corresponding to "summer".

In the case of a gap, when clocks jump forward, there is no valid offset. Instead, the local date-time is adjusted to be later by the length of the gap. For a typical one hour daylight savings change, the local date-time will be moved one hour later into the offset typically corresponding to "summer".

Parameters:

localDateTime - the local date-time, not null

zone - the time-zone, not null

Returns:

the zoned date-time, not null

of

```
public static ZonedDateTime of(int year,
                               int month,
                               int dayOfMonth,
                               int hour,
                               int minute,
                               int second,
                               int nanoOfSecond,
                               ZoneId zone)
```

Obtains an instance of `ZonedDateTime` from a year, month, day, hour, minute, second, nanosecond and time-zone.

This creates a zoned date-time matching the local date-time of the seven specified fields as closely as possible. Time-zone rules, such as daylight savings, mean that not every local date-time is valid for the specified zone, thus the local date-time may be adjusted.

The local date-time is resolved to a single instant on the time-line. This is achieved by finding a valid offset from UTC/Greenwich for the local date-time as defined by the `rules` of the zone ID.

In most cases, there is only one valid offset for a local date-time. In the case of an overlap, when clocks are set back, there are two valid offsets. This method uses the earlier offset typically corresponding to "summer".

In the case of a gap, when clocks jump forward, there is no valid offset. Instead, the local date-time is adjusted to be later by the length of the gap. For a typical one hour daylight savings change, the local date-time will be moved one hour later into the offset typically corresponding to "summer".

This method exists primarily for writing test cases. Non test-code will typically use other methods to create an offset time. `LocalDateTime` has five additional convenience variants of the equivalent factory method taking fewer arguments. They are not provided here to reduce the footprint of the API.

Parameters:

`year` - the year to represent, from MIN_YEAR to MAX_YEAR

`month` - the month-of-year to represent, from 1 (January) to 12 (December)

`dayOfMonth` - the day-of-month to represent, from 1 to 31

`hour` - the hour-of-day to represent, from 0 to 23

`minute` - the minute-of-hour to represent, from 0 to 59

`second` - the second-of-minute to represent, from 0 to 59

`nanoOfSecond` - the nano-of-second to represent, from 0 to 999,999,999

`zone` - the time-zone, not null

Returns:

the zoned date-time, not null

Throws:

`DateTimeException` - if the value of any field is out of range, or if the day-of-month is invalid for the month-year

ofLocal

```
public static ZonedDateTime ofLocal(LocalDateTime localDateTime,
                                    ZoneId zone,
                                    ZoneOffset preferredOffset)
```

Obtains an instance of `ZonedDateTime` from a local date-time using the preferred offset if possible.

The local date-time is resolved to a single instant on the time-line. This is achieved by finding a valid offset from UTC/Greenwich for the local date-time as defined by the `rules` of the zone ID.

In most cases, there is only one valid offset for a local date-time. In the case of an overlap, where clocks are set back, there are two valid offsets. If the preferred offset is one of the valid offsets then it is used. Otherwise the earlier valid offset is used, typically corresponding to "summer".

In the case of a gap, where clocks jump forward, there is no valid offset. Instead, the local date-time is adjusted to be later by the length of the gap. For a typical one hour daylight savings change, the local date-time will be moved one hour later into the offset typically corresponding to "summer".

Parameters:

`localDateTime` - the local date-time, not null

`zone` - the time-zone, not null

`preferredOffset` - the zone offset, null if no preference

Returns:

the zoned date-time, not null

ofInstant

```
public static ZonedDateTime ofInstant(Instant instant,
                                      ZoneId zone)
```

Obtains an instance of `ZonedDateTime` from an `Instant`.

This
creates a zoned date-time with the same instant as that specified. Calling `ChronoZonedDateTime.toInstant()` will return an instant equal to the one used here.

Converting an instant to a zoned date-time is simple as there is only one valid offset for each instant.

Parameters:

`instant` - the instant to create the date-time from, not null

zone - the time-zone, not null

Returns:

the zoned date-time, not null

Throws:

DateTimeException - if the result exceeds the supported range

ofInstant

```
public static ZonedDateTime ofInstant(LocalDateTime localDateTime,
                                      ZoneOffset offset,
                                      ZoneId zone)
```

Obtains an instance of ZonedDateTime from the instant formed by combining the local date-time and offset.

This creates a zoned date-time by combining the LocalDateTime and ZoneOffset.
This combination uniquely specifies an instant without ambiguity.

Converting an instant to a zoned date-time is simple as there is only one valid offset for each instant. If the valid offset is different to the offset specified, then the date-time and offset of the zoned date-time will differ from those specified.

If the ZoneId to be used is a ZoneOffset, this method is equivalent to of(LocalDateTime, ZoneId).

Parameters:

localDateTime - the local date-time, not null

offset - the zone offset, not null

zone - the time-zone, not null

Returns:

the zoned date-time, not null

ofStrict

```
public static ZonedDateTime ofStrict(LocalDateTime localDateTime,
                                     ZoneOffset offset,
                                     ZoneId zone)
```

Obtains an instance of ZonedDateTime strictly validating the combination of local date-time, offset and zone ID.

This creates a zoned date-time ensuring that the offset is valid for the local date-time according to the rules of the specified zone. If the offset is invalid, an exception is thrown.

Parameters:

`localDateTime` - the local date-time, not null

`offset` - the zone offset, not null

`zone` - the time-zone, not null

Returns:

the zoned date-time, not null

Throws:

`DateTimeException` - if the combination of arguments is invalid

from

```
public static ZonedDateTime from(TemporalAccessor temporal)
```

Obtains an instance of `ZonedDateTime` from a temporal object.

This obtains a zoned date-time based on the specified temporal. A `TemporalAccessor` represents an arbitrary set of date and time information, which this factory converts to an instance of `ZonedDateTime`.

The conversion will first obtain a `ZoneId` from the temporal object, falling back to a `ZoneOffset` if necessary. It will then try to obtain an `Instant`, falling back to a `LocalDateTime` if necessary. The result will be either the combination of `ZoneId` or `ZoneOffset` with `Instant` or `LocalDateTime`. Implementations are permitted to perform optimizations such as accessing those fields that are equivalent to the relevant objects.

This method matches the signature of the functional interface `TemporalQuery` allowing it to be used as a query via method reference, `ZonedDateTime::from`.

Parameters:

`temporal` - the temporal object to convert, not null

Returns:

the zoned date-time, not null

Throws:

`DateTimeException` - if unable to convert to an `ZonedDateTime`

parse

```
public static ZonedDateTime parse(CharSequence text)
```

Obtains an instance of ZonedDateTime from a text string such as 2007-12-03T10:15:30+01:00[Europe/Paris].

The string must represent a valid date-time and is parsed using DateTimeFormatter.ISO_ZONED_DATE_TIME.

Parameters:

text - the text to parse such as "2007-12-03T10:15:30+01:00[Europe/Paris]", not null

Returns:

the parsed zoned date-time, not null

Throws:

DateTimeParseException - if the text cannot be parsed

parse

```
public static ZonedDateTime parse(CharSequence text,
                                  DateTimeFormatter formatter)
```

Obtains an instance of ZonedDateTime from a text string using a specific formatter.

The text is parsed using the formatter, returning a date-time.

Parameters:

text - the text to parse, not null

formatter - the formatter to use, not null

Returns:

the parsed zoned date-time, not null

Throws:

DateTimeParseException - if the text cannot be parsed

isSupported

```
public boolean isSupported(TemporalField field)
```

Checks if the specified field is supported.

This checks if this date-time can be queried for the specified field. If false, then calling the range, get and with(TemporalField, long) methods will throw an exception.

If the field is a ChronoField then the query is implemented here. The supported fields are:

- NANO_OF_SECOND

- NANO_OF_DAY
- MICRO_OF_SECOND
- MICRO_OF_DAY
- MILLI_OF_SECOND
- MILLI_OF_DAY
- SECOND_OF_MINUTE
- SECOND_OF_DAY
- MINUTE_OF_HOUR
- MINUTE_OF_DAY
- HOUR_OF_AMPM
- CLOCK_HOUR_OF_AMPM
- HOUR_OF_DAY
- CLOCK_HOUR_OF_DAY
- AMPM_OF_DAY
- DAY_OF_WEEK
- ALIGNED_DAY_OF_WEEK_IN_MONTH
- ALIGNED_DAY_OF_WEEK_IN_YEAR
- DAY_OF_MONTH
- DAY_OF_YEAR
- EPOCH_DAY
- ALIGNED_WEEK_OF_MONTH
- ALIGNED_WEEK_OF_YEAR
- MONTH_OF_YEAR
- PROLEPTIC_MONTH
- YEAR_OF_ERA
- YEAR
- ERA
- INSTANT_SECONDS
- OFFSET_SECONDS

All other `ChronoField` instances will return false.

If the field is not a `ChronoField`, then the result of this method is obtained by invoking `TemporalField.isSupportedBy(TemporalAccessor)` passing `this` as the argument. Whether the field is supported is determined by the field.

Specified by:

`isSupported` in interface `ChronoZonedDateTime<LocalDate>`

Specified by:

`isSupported` in interface `TemporalAccessor`

Parameters:

`field` - the field to check, null returns false

Returns:

true if the field is supported on this date-time, false if not

isSupported

`public boolean isSupported(TemporalUnit unit)`

Checks if the specified unit is supported.

This checks if the specified unit can be added to, or subtracted from, this date-time. If false, then calling the `plus(long, TemporalUnit)` and `minus` methods will throw an exception.

If the unit is a `ChronoUnit` then the query is implemented here. The supported units are:

- `NANOS`
- `MICROS`
- `MILLIS`
- `SECONDS`
- `MINUTES`
- `HOURS`
- `HALF_DAYS`
- `DAYS`
- `WEEKS`
- `MONTHS`
- `YEARS`
- `DECADES`
- `CENTURIES`
- `MILLENNIA`
- `ERAS`

All other `ChronoUnit` instances will return false.

If the unit is not a `ChronoUnit`, then the result of this method is obtained by invoking `TemporalUnit.isSupportedBy(Temporal)` passing `this` as the argument. Whether the unit is supported is determined by the unit.

Specified by:

`isSupported` in interface `ChronoZonedDateTime`<`LocalDate`>

Specified by:

`isSupported` in interface `Temporal`

Parameters:

`unit` - the unit to check, null returns false

Returns:

true if the unit can be added/subtracted, false if not

range

`public ValueRange range(TemporalField field)`

Gets the range of valid values for the specified field.

The range object expresses the minimum and maximum valid values for a field. This date-time is used to enhance the accuracy of the returned range. If it is not possible to return the range, because the field is not supported or for some other reason, an exception is thrown.

If the field is a `ChronoField` then the query is implemented here. The `supported fields` will return appropriate range instances. All other `ChronoField` instances will throw an `UnsupportedTemporalTypeException`.

If the field is not a `ChronoField`, then the result of this method is obtained by invoking `TemporalField.rangeRefinedBy(TemporalAccessor)` passing `this` as the argument. Whether the range can be obtained is determined by the field.

Specified by:

`range` in interface `TemporalAccessor`

Parameters:

`field` - the field to query the range for, not null

Returns:

the range of valid values for the field, not null

Throws:

`DateTimeException` - if the range for the field cannot be obtained

`UnsupportedTemporalTypeException` - if the field is not supported

get

`public int get(TemporalField field)`

Gets the value of the specified field from this date-time as an `int`.

This queries this date-time for the value of the specified field. The returned value will always be within the valid range of values for the field. If it is not possible to return the value, because the field is not supported or for some other reason, an exception is thrown.

If the field is a `ChronoField` then the query is implemented here. The `supported fields` will return valid values based on this date-time, except `NANO_OF_DAY`, `MICRO_OF_DAY`, `EPOCH_DAY`, `PROLEPTIC_MONTH` and `INSTANT_SECONDS` which are too large to fit in an `int` and throw an

UnsupportedTemporalTypeException. All other ChronoField instances will throw an UnsupportedTemporalTypeException.

If the field is not a ChronoField, then the result of this method is obtained by invoking TemporalField.getFrom(TemporalAccessor) passing this as the argument. Whether the value can be obtained, and what the value represents, is determined by the field.

Specified by:

get in interface TemporalAccessor

Parameters:

field - the field to get, not null

Returns:

the value for the field

Throws:

DateTimeException - if a value for the field cannot be obtained or the value is outside the range of valid values for the field

UnsupportedTemporalTypeException - if the field is not supported or the range of values exceeds an int

ArithmeticException - if numeric overflow occurs

getLong

```
public long getLong(TemporalField field)
```

Gets the value of the specified field from this date-time as a long.

This queries this date-time for the value of the specified field. If it is not possible to return the value, because the field is not supported or for some other reason, an exception is thrown.

If the field is a ChronoField then the query is implemented here. The supported fields will return valid values based on this date-time. All other ChronoField instances will throw an UnsupportedTemporalTypeException.

If the field is not a ChronoField, then the result of this method is obtained by invoking TemporalField.getFrom(TemporalAccessor) passing this as the argument. Whether the value can be obtained, and what the value represents, is determined by the field.

Specified by:

getLong in interface ChronoZonedDateTime<LocalDate>

Specified by:

getLong in interface TemporalAccessor

Parameters:

`field` - the field to get, not null

Returns:

the value for the field

Throws:

`DateTimeException` - if a value for the field cannot be obtained

`UnsupportedTemporalTypeException` - if the field is not supported

`ArithmeticException` - if numeric overflow occurs

getOffset

`public ZoneOffset getOffset()`

Gets the zone offset, such as '+01:00'.

This is the offset of the local date-time from UTC/Greenwich.

Specified by:

`getOffset` in interface `ChronoZonedDateTime<LocalDate>`

Returns:

the zone offset, not null

withEarlierOffsetAtOverlap

`public ZonedDateTime withEarlierOffsetAtOverlap()`

Returns a copy of this date-time changing the zone offset to the earlier of the two valid offsets at a local time-line overlap.

This method only has any effect when the local time-line overlaps, such as at an autumn daylight savings cutover. In this scenario, there are two valid offsets for the local date-time. Calling this method will return a zoned date-time with the earlier of the two selected.

If this method is called when it is not an overlap, `this` is returned.

This instance is immutable and unaffected by this method call.

Specified by:

`withEarlierOffsetAtOverlap` in interface `ChronoZonedDateTime<LocalDate>`

Returns:

a `ZonedDateTime` based on this date-time with the earlier offset, not null

withLaterOffsetAtOverlap

`public ZonedDateTime withLaterOffsetAtOverlap()`

Returns a copy of this date-time changing the zone offset to the later of the two valid offsets at a local time-line overlap.

This method only has any effect when the local time-line overlaps, such as at an autumn daylight savings cutover. In this scenario, there are two valid offsets for the local date-time. Calling this method will return a zoned date-time with the later of the two selected.

If this method is called when it is not an overlap, `this` is returned.

This instance is immutable and unaffected by this method call.

Specified by:

`withLaterOffsetAtOverlap` in interface `ChronoZonedDateTime<LocalDate>`

Returns:

a `ZonedDateTime` based on this date-time with the later offset, not null

getZone

`public ZoneId getZone()`

Gets the time-zone, such as 'Europe/Paris'.

This returns the zone ID. This identifies the time-zone `rules` that determine when and how the offset from UTC/Greenwich changes.

The zone ID may be same as the offset. If this is true, then any future calculations, such as addition or subtraction, have no complex edge cases due to time-zone rules. See also `withFixedOffsetZone()`.

Specified by:

`getZone` in interface `ChronoZonedDateTime<LocalDate>`

Returns:

the time-zone, not null

withZoneSameLocal

`public ZonedDateTime withZoneSameLocal(ZoneId zone)`

Returns a copy of this date-time with a different time-zone, retaining the local date-time if possible.

This method changes the time-zone and retains the local date-time. The local date-time is only changed if it is invalid for the new zone, determined using the same approach as `ofLocal(LocalDateTime, ZoneId, ZoneOffset)`.

To change the zone and adjust the local date-time, use `withZoneSameInstant(ZoneId)`.

This instance is immutable and unaffected by this method call.

Specified by:

`withZoneSameLocal` in interface `ChronoZonedDateTime<LocalDate>`

Parameters:

`zone` - the time-zone to change to, not null

Returns:

a `ZonedDateTime` based on this date-time with the requested zone, not null

withZoneSameInstant

```
public ZonedDateTime withZoneSameInstant(ZoneId zone)
```

Returns a copy of this date-time with a different time-zone, retaining the instant.

This method changes the time-zone and retains the instant. This normally results in a change to the local date-time.

This method is based on retaining the same instant, thus gaps and overlaps in the local time-line have no effect on the result.

To change the offset while keeping the local time, use `withZoneSameLocal(ZoneId)`.

Specified by:

`withZoneSameInstant` in interface `ChronoZonedDateTime<LocalDate>`

Parameters:

`zone` - the time-zone to change to, not null

Returns:

a `ZonedDateTime` based on this date-time with the requested zone, not null

Throws:

`DateTimeException` - if the result exceeds the supported date range

withFixedOffsetZone

```
public ZonedDateTime withFixedOffsetZone()
```

Returns a copy of this date-time with the zone ID set to the offset.

This returns a zoned date-time where the zone ID is the same as `getOffset()`. The local date-time, offset and instant of the result will be the same as in this date-time.

Setting the date-time to a fixed single offset means that any future calculations, such as addition or subtraction, have no complex edge cases due to time-zone rules. This might also be useful when sending a zoned date-time across a network, as most protocols, such as ISO-8601, only handle offsets, and not region-based zone IDs.

This is equivalent to `ZonedDateTime.of(zdt.toLocalDateTime(), zdt.getOffset())`.

Returns:

a `ZonedDateTime` with the zone ID set to the offset, not null

toLocalDateTime

```
public LocalDateTime toLocalDateTime()
```

Gets the `LocalDateTime` part of this date-time.

This returns a `LocalDateTime` with the same year, month, day and time as this date-time.

Specified by:

`toLocalDateTime` in interface `ChronoZonedDateTime<LocalDate>`

Returns:

the local date-time part of this date-time, not null

toLocalDate

```
public LocalDate toLocalDate()
```

Gets the `LocalDate` part of this date-time.

This returns a `LocalDate` with the same year, month and day as this date-time.

Specified by:

`toLocalDate` in interface `ChronoZonedDateTime<LocalDate>`

Returns:

the date part of this date-time, not null

getYear

```
public int getYear()
```

Gets the year field.

This method returns the primitive `int` value for the year.

The year returned by this method is proleptic as per `get(YEAR)`. To obtain the year-of-era, use `get(YEAR_OF_ERA)`.

Returns:

the year, from MIN_YEAR to MAX_YEAR

getMonthValue

```
public int getMonthValue()
```

Gets the month-of-year field from 1 to 12.

This method returns the month as an `int` from 1 to 12. Application code is frequently clearer if the enum `Month` is used by calling `getMonth()`.

Returns:

the month-of-year, from 1 to 12

See Also:

`getMonth()`

getMonth

```
public Month getMonth()
```

Gets the month-of-year field using the `Month` enum.

This method returns the enum `Month` for the month. This avoids confusion as to what `int` values mean. If you need access to the primitive `int` value then the enum provides the `int value`.

Returns:

the month-of-year, not null

See Also:

`getMonthValue()`

getDayOfMonth

```
public int getDayOfMonth()
```

Gets the day-of-month field.

This method returns the primitive `int` value for the day-of-month.

Returns:

the day-of-month, from 1 to 31

getDayOfYear

`public int getDayOfYear()`

Gets the day-of-year field.

This method returns the primitive `int` value for the day-of-year.

Returns:

the day-of-year, from 1 to 365, or 366 in a leap year

getDayOfWeek

`public DayOfWeek getDayOfWeek()`

Gets the day-of-week field, which is an enum `DayOfWeek`.

This method returns the enum `DayOfWeek` for the day-of-week. This avoids confusion as to what `int` values mean. If you need access to the primitive `int` value then the enum provides the `int value`.

Additional information can be obtained from the `DayOfWeek`. This includes textual names of the values.

Returns:

the day-of-week, not null

toLocalTime

`public LocalTime toLocalTime()`

Gets the `LocalTime` part of this date-time.

This returns a `LocalTime` with the same hour, minute, second and nanosecond as this date-time.

Specified by:

`toLocalTime` in interface `ChronoZonedDateTime<LocalDate>`

Returns:

the time part of this date-time, not null

getHour

`public int getHour()`

Gets the hour-of-day field.

Returns:

the hour-of-day, from 0 to 23

getMinute

```
public int getMinute()
```

Gets the minute-of-hour field.

Returns:

the minute-of-hour, from 0 to 59

getSecond

```
public int getSecond()
```

Gets the second-of-minute field.

Returns:

the second-of-minute, from 0 to 59

getNano

```
public int getNano()
```

Gets the nano-of-second field.

Returns:

the nano-of-second, from 0 to 999,999,999

with

```
public ZonedDateTime with(TemporalAdjuster adjuster)
```

Returns an adjusted copy of this date-time.

This returns a `ZonedDateTime`, based on this one, with the date-time adjusted. The adjustment takes place using the specified adjuster strategy object. Read the documentation of the adjuster to understand what adjustment will be made.

A simple adjuster might simply set the one of the fields, such as the year field. A more complex adjuster might set the date to the last day of the month. A selection of common adjustments is provided in `TemporalAdjusters`. These include finding the "last day of the month" and "next Wednesday". Key date-time classes also implement the `TemporalAdjuster` interface, such as `Month` and `MonthDay`. The adjuster is responsible for handling special cases, such as the varying lengths of month and leap years.

For example this code returns a date on the last day of July:

```
import static java.time.Month.*;
import static java.time.temporal.TemporalAdjusters.*;

result = zonedDateTime.with(JULY).with(lastDayOfMonth());
```

The classes `LocalDate` and `LocalTime` implement `TemporalAdjuster`, thus this method can be used to change the date, time or offset:

```
result = zonedDateTime.with(date);
result = zonedDateTime.with(time);
```

`ZoneOffset` also implements `TemporalAdjuster` however using it as an argument typically has no effect. The offset of a `ZonedDateTime` is controlled primarily by the time-zone. As such, changing the offset does not generally make sense, because there is only one valid offset for the local date-time and zone. If the zoned date-time is in a daylight savings overlap, then the offset is used to switch between the two valid offsets. In all other cases, the offset is ignored.

The result of this method is obtained by invoking the `TemporalAdjuster.adjustInto(Temporal)` method on the specified adjuster passing `this` as the argument.

This instance is immutable and unaffected by this method call.

Specified by:

`with` in interface `ChronoZonedDateTime<LocalDate>`

Specified by:

`with` in interface `Temporal`

Parameters:

`adjuster` - the adjuster to use, not null

Returns:

a `ZonedDateTime` based on `this` with the adjustment made, not null

Throws:

`DateTimeException` - if the adjustment cannot be made

`ArithmeticException` - if numeric overflow occurs

with

```
public ZonedDateTime with(TemporalField field,
                          long newValue)
```

Returns a copy of this date-time with the specified field set to a new value.

This returns a `ZonedDateTime`, based on this one, with the value for the specified field changed. This can be used to change any supported field, such as the year, month or day-of-month. If it is not possible to set the value, because the field is not supported or for some other reason, an exception is thrown.

In some cases, changing the specified field can cause the resulting date-time to become invalid, such as changing the month from 31st January to February would make the day-of-month invalid. In cases like this, the field is responsible for resolving the date. Typically it will choose the previous valid date, which would be the last valid day of February in this example.

If the field is a `ChronoField` then the adjustment is implemented here.

The `INSTANT_SECONDS` field will return a date-time with the specified instant. The zone and nano-of-second are unchanged. The result will have an offset derived from the new instant and original zone. If the new instant value is outside the valid range then a `DateTimeException` will be thrown.

The `OFFSET_SECONDS` field will typically be ignored. The offset of a `ZonedDateTime` is controlled primarily by the time-zone. As such, changing the offset does not generally make sense, because there is only one valid offset for the local date-time and zone. If the zoned date-time is in a daylight savings overlap, then the offset is used to switch between the two valid offsets. In all other cases, the offset is ignored. If the new offset value is outside the valid range then a `DateTimeException` will be thrown.

The other `supported fields` will behave as per the matching method on `LocalDateTime`. The zone is not part of the calculation and will be unchanged. When converting back to `ZonedDateTime`, if the local date-time is in an overlap, then the offset will be retained if possible, otherwise the earlier offset will be used. If in a gap, the local date-time will be adjusted forward by the length of the gap.

All other `ChronoField` instances will throw an `UnsupportedTemporalTypeException`.

If the field is not a `ChronoField`, then the result of this method is obtained by invoking `TemporalField.adjustInto(Temporal, long)` passing `this` as the argument. In this case, the field determines whether and how to adjust the instant.

This instance is immutable and unaffected by this method call.

Specified by:

`with` in interface `ChronoZonedDateTime`<`LocalDate`>

Specified by:

`with` in interface `Temporal`

Parameters:

`field` - the field to set in the result, not null

`newValue` - the new value of the field in the result

Returns:

a `ZonedDateTime` based on `this` with the specified field set, not null

Throws:

`DateTimeException` - if the field cannot be set

`UnsupportedTemporalTypeException` - if the field is not supported

`ArithmeticException` - if numeric overflow occurs

withYear

`public ZonedDateTime withYear(int year)`

Returns a copy of this `ZonedDateTime` with the year altered.

This operates on the local time-line, `changing the year` of the local date-time. This is then converted back to a `ZonedDateTime`, using the zone ID to obtain the offset.

When converting back to `ZonedDateTime`, if the local date-time is in an overlap, then the offset will be retained if possible, otherwise the earlier offset will be used. If in a gap, the local date-time will be adjusted forward by the length of the gap.

This instance is immutable and unaffected by this method call.

Parameters:

`year` - the year to set in the result, from MIN_YEAR to MAX_YEAR

Returns:

a `ZonedDateTime` based on this date-time with the requested year, not null

Throws:

`DateTimeException` - if the year value is invalid

withMonth

`public ZonedDateTime withMonth(int month)`

Returns a copy of this `ZonedDateTime` with the month-of-year altered.

This operates on the local time-line, changing the month of the local date-time. This is then converted back to a ZonedDateTime, using the zone ID to obtain the offset.

When converting back to ZonedDateTime, if the local date-time is in an overlap, then the offset will be retained if possible, otherwise the earlier offset will be used. If in a gap, the local date-time will be adjusted forward by the length of the gap.

This instance is immutable and unaffected by this method call.

Parameters:

month - the month-of-year to set in the result, from 1 (January) to 12 (December)

Returns:

a ZonedDateTime based on this date-time with the requested month, not null

Throws:

DateTimeException - if the month-of-year value is invalid

withDayOfMonth

```
public ZonedDateTime withDayOfMonth(int dayOfMonth)
```

Returns a copy of this ZonedDateTime with the day-of-month altered.

This operates on the local time-line, changing the day-of-month of the local date-time. This is then converted back to a ZonedDateTime, using the zone ID to obtain the offset.

When converting back to ZonedDateTime, if the local date-time is in an overlap, then the offset will be retained if possible, otherwise the earlier offset will be used. If in a gap, the local date-time will be adjusted forward by the length of the gap.

This instance is immutable and unaffected by this method call.

Parameters:

dayOfMonth - the day-of-month to set in the result, from 1 to 28-31

Returns:

a ZonedDateTime based on this date-time with the requested day, not null

Throws:

DateTimeException - if the day-of-month value is invalid, or if the day-of-month is invalid for the month-year

withDayOfYear

```
public ZonedDateTime withDayOfYear(int dayOfYear)
```

Returns a copy of this `ZonedDateTime` with the day-of-year altered.

This operates on the local time-line, `changing the day-of-year` of the local date-time. This is then converted back to a `ZonedDateTime`, using the zone ID to obtain the offset.

When converting back to `ZonedDateTime`, if the local date-time is in an overlap, then the offset will be retained if possible, otherwise the earlier offset will be used. If in a gap, the local date-time will be adjusted forward by the length of the gap.

This instance is immutable and unaffected by this method call.

Parameters:

`dayOfYear` - the day-of-year to set in the result, from 1 to 365-366

Returns:

a `ZonedDateTime` based on this date with the requested day, not null

Throws:

`DateTimeException` - if the day-of-year value is invalid, or if the day-of-year is invalid for the year

withHour

```
public ZonedDateTime withHour(int hour)
```

Returns a copy of this `ZonedDateTime` with the hour-of-day altered.

This operates on the local time-line, changing the time of the local date-time. This is then converted back to a `ZonedDateTime`, using the zone ID to obtain the offset.

When converting back to `ZonedDateTime`, if the local date-time is in an overlap, then the offset will be retained if possible, otherwise the earlier offset will be used. If in a gap, the local date-time will be adjusted forward by the length of the gap.

This instance is immutable and unaffected by this method call.

Parameters:

`hour` - the hour-of-day to set in the result, from 0 to 23

Returns:

a `ZonedDateTime` based on this date-time with the requested hour, not null

Throws:

`DateTimeException` - if the hour value is invalid

withMinute

```
public ZonedDateTime withMinute(int minute)
```

Returns a copy of this `ZonedDateTime` with the minute-of-hour altered.

This operates on the local time-line, changing the time of the local date-time. This is then converted back to a `ZonedDateTime`, using the zone ID to obtain the offset.

When converting back to `ZonedDateTime`, if the local date-time is in an overlap, then the offset will be retained if possible, otherwise the earlier offset will be used. If in a gap, the local date-time will be adjusted forward by the length of the gap.

This instance is immutable and unaffected by this method call.

Parameters:

`minute` - the minute-of-hour to set in the result, from 0 to 59

Returns:

a `ZonedDateTime` based on this date-time with the requested minute, not null

Throws:

`DateTimeException` - if the minute value is invalid

withSecond

```
public ZonedDateTime withSecond(int second)
```

Returns a copy of this `ZonedDateTime` with the second-of-minute altered.

This operates on the local time-line, changing the time of the local date-time. This is then converted back to a `ZonedDateTime`, using the zone ID to obtain the offset.

When converting back to `ZonedDateTime`, if the local date-time is in an overlap, then the offset will be retained if possible, otherwise the earlier offset will be used. If in a gap, the local date-time will be adjusted forward by the length of the gap.

This instance is immutable and unaffected by this method call.

Parameters:

`second` - the second-of-minute to set in the result, from 0 to 59

Returns:

a `ZonedDateTime` based on this date-time with the requested second, not null

Throws:

`DateTimeException` - if the second value is invalid

withNano

```
public ZonedDateTime withNano(int nanoOfSecond)
```

Returns a copy of this `ZonedDateTime` with the nano-of-second altered.

This operates on the local time-line, changing the time of the local date-time. This is then converted back to a `ZonedDateTime`, using the zone ID to obtain the offset.

When converting back to `ZonedDateTime`, if the local date-time is in an overlap, then the offset will be retained if possible, otherwise the earlier offset will be used. If in a gap, the local date-time will be adjusted forward by the length of the gap.

This instance is immutable and unaffected by this method call.

Parameters:

`nanoOfSecond` - the nano-of-second to set in the result, from 0 to 999,999,999

Returns:

a `ZonedDateTime` based on this date-time with the requested nanosecond, not null

Throws:

`DateTimeException` - if the nano value is invalid

truncatedTo

```
public ZonedDateTime truncatedTo(TemporalUnit unit)
```

Returns a copy of this `ZonedDateTime` with the time truncated.

Truncation returns a copy of the original date-time with fields smaller than the specified unit set to zero. For example, truncating with the `minutes` unit will set the second-of-minute and nano-of-second field to zero.

The unit must have a duration that divides into the length of a standard day without remainder. This includes all supplied time units on `ChronoUnit` and `DAYS`. Other units throw an exception.

This operates on the local time-line, `truncating` the underlying local date-time. This is then converted back to a `ZonedDateTime`, using the zone ID to obtain the offset.

When converting back to `ZonedDateTime`, if the local date-time is in an overlap, then the offset will be retained if possible, otherwise the earlier offset will be used. If in a gap, the local date-time will be adjusted forward by the length of the gap.

This instance is immutable and unaffected by this method call.

Parameters:

`unit` - the unit to truncate to, not null

Returns:

a `ZonedDateTime` based on this date-time with the time truncated, not null

Throws:

`DateTimeException` - if unable to truncate

`UnsupportedTemporalTypeException` - if the unit is not supported

plus

```
public ZonedDateTime plus(TemporalAmount amountToAdd)
```

Returns a copy of this date-time with the specified amount added.

This returns a `ZonedDateTime`, based on this one, with the specified amount added. The amount is typically `Period` or `Duration` but may be any other type implementing the `TemporalAmount` interface.

The calculation is delegated to the amount object by calling `TemporalAmount.addTo(Temporal)`. The amount implementation is free to implement the addition in any way it wishes, however it typically calls back to `plus(long, TemporalUnit)`. Consult the documentation of the amount implementation to determine if it can be successfully added.

This instance is immutable and unaffected by this method call.

Specified by:

`plus` in interface `ChronoZonedDateTime<LocalDate>`

Specified by:

`plus` in interface `Temporal`

Parameters:

`amountToAdd` - the amount to add, not null

Returns:

a ZonedDateTime based on this date-time with the addition made, not null

Throws:

DateTimeException - if the addition cannot be made

ArithmeticException - if numeric overflow occurs

plus

```
public ZonedDateTime plus(long amountToAdd,
                          TemporalUnit unit)
```

Returns a copy of this date-time with the specified amount added.

This returns a ZonedDateTime, based on this one, with the amount in terms of the unit added. If it is not possible to add the amount, because the unit is not supported or for some other reason, an exception is thrown.

If the field is a ChronoUnit then the addition is implemented here. The zone is not part of the calculation and will be unchanged in the result. The calculation for date and time units differ.

Date units operate on the local time-line. The period is first added to the local date-time, then converted back to a zoned date-time using the zone ID. The conversion uses ofLocal(LocalDateTime, ZoneId, ZoneOffset) with the offset before the addition.

Time units operate on the instant time-line. The period is first added to the local date-time, then converted back to a zoned date-time using the zone ID. The conversion uses ofInstant(LocalDateTime, ZoneOffset, ZoneId) with the offset before the addition.

If the field is not a ChronoUnit, then the result of this method is obtained by invoking TemporalUnit.addTo(Temporal, long) passing this as the argument. In this case, the unit determines whether and how to perform the addition.

This instance is immutable and unaffected by this method call.

Specified by:

plus in interface ChronoZonedDateTime<LocalDate>

Specified by:

plus in interface Temporal

Parameters:

amountToAdd - the amount of the unit to add to the result, may be negative

unit - the unit of the amount to add, not null

Returns:

a ZonedDateTime based on this date-time with the specified amount added, not null

Throws:

DateTimeException - if the addition cannot be made

UnsupportedTemporalTypeException - if the unit is not supported

ArithmeticException - if numeric overflow occurs

plusYears

```
public ZonedDateTime plusYears(long years)
```

Returns a copy of this ZonedDateTime with the specified number of years added.

This operates on the local time-line, adding years to the local date-time. This is then converted back to a ZonedDateTime, using the zone ID to obtain the offset.

When converting back to ZonedDateTime, if the local date-time is in an overlap, then the offset will be retained if possible, otherwise the earlier offset will be used. If in a gap, the local date-time will be adjusted forward by the length of the gap.

This instance is immutable and unaffected by this method call.

Parameters:

years - the years to add, may be negative

Returns:

a ZonedDateTime based on this date-time with the years added, not null

Throws:

DateTimeException - if the result exceeds the supported date range

plusMonths

```
public ZonedDateTime plusMonths(long months)
```

Returns a copy of this ZonedDateTime with the specified number of months added.

This operates on the local time-line, adding months to the local date-time. This is then converted back to a ZonedDateTime, using the zone ID to obtain the offset.

When converting back to ZonedDateTime, if the local date-time is in an overlap, then the offset will be retained if possible, otherwise the earlier offset

will be used. If in a gap, the local date-time will be adjusted forward by the length of the gap.

This instance is immutable and unaffected by this method call.

Parameters:

`months` - the months to add, may be negative

Returns:

a `ZonedDateTime` based on this date-time with the months added, not null

Throws:

`DateTimeException` - if the result exceeds the supported date range

plusWeeks

```
public ZonedDateTime plusWeeks(long weeks)
```

Returns a copy of this `ZonedDateTime` with the specified number of weeks added.

This operates on the local time-line, `adding weeks` to the local date-time. This is then converted back to a `ZonedDateTime`, using the zone ID to obtain the offset.

When converting back to `ZonedDateTime`, if the local date-time is in an overlap, then the offset will be retained if possible, otherwise the earlier offset will be used. If in a gap, the local date-time will be adjusted forward by the length of the gap.

This instance is immutable and unaffected by this method call.

Parameters:

`weeks` - the weeks to add, may be negative

Returns:

a `ZonedDateTime` based on this date-time with the weeks added, not null

Throws:

`DateTimeException` - if the result exceeds the supported date range

plusDays

```
public ZonedDateTime plusDays(long days)
```

Returns a copy of this `ZonedDateTime` with the specified number of days added.

This operates on the local time-line, `adding days` to the local date-time. This is then converted back to a `ZonedDateTime`, using the zone ID to obtain the offset.

When converting back to `ZonedDateTime`, if the local date-time is in an overlap, then the offset will be retained if possible, otherwise the earlier offset will be used. If in a gap, the local date-time will be adjusted forward by the length of the gap.

This instance is immutable and unaffected by this method call.

Parameters:

`days` - the days to add, may be negative

Returns:

a `ZonedDateTime` based on this date-time with the days added, not null

Throws:

`DateTimeException` - if the result exceeds the supported date range

plusHours

```
public ZonedDateTime plusHours(long hours)
```

Returns a copy of this `ZonedDateTime` with the specified number of hours added.

This operates on the instant time-line, such that adding one hour will always be a duration of one hour later. This may cause the local date-time to change by an amount other than one hour. Note that this is a different approach to that used by days, months and years, thus adding one day is not the same as adding 24 hours.

For example, consider a time-zone, such as 'Europe/Paris', where the Autumn DST cutover means that the local times 02:00 to 02:59 occur twice changing from offset +02:00 in summer to +01:00 in winter.

- Adding one hour to 01:30+02:00 will result in 02:30+02:00 (both in summer time)
- Adding one hour to 02:30+02:00 will result in 02:30+01:00 (moving from summer to winter time)
- Adding one hour to 02:30+01:00 will result in 03:30+01:00 (both in winter time)
- Adding three hours to 01:30+02:00 will result in 03:30+01:00 (moving from summer to winter time)

This instance is immutable and unaffected by this method call.

Parameters:

`hours` - the hours to add, may be negative

Returns:

a `ZonedDateTime` based on this date-time with the hours added, not null

Throws:

`DateTimeException` - if the result exceeds the supported date range

plusMinutes

```
public ZonedDateTime plusMinutes(long minutes)
```

Returns a copy of this `ZonedDateTime` with the specified number of minutes added.

This operates on the instant time-line, such that adding one minute will always be a duration of one minute later. This may cause the local date-time to change by an amount other than one minute. Note that this is a different approach to that used by days, months and years.

This instance is immutable and unaffected by this method call.

Parameters:

`minutes` - the minutes to add, may be negative

Returns:

a `ZonedDateTime` based on this date-time with the minutes added, not null

Throws:

`DateTimeException` - if the result exceeds the supported date range

plusSeconds

```
public ZonedDateTime plusSeconds(long seconds)
```

Returns a copy of this `ZonedDateTime` with the specified number of seconds added.

This operates on the instant time-line, such that adding one second will always be a duration of one second later. This may cause the local date-time to change by an amount other than one second. Note that this is a different approach to that used by days, months and years.

This instance is immutable and unaffected by this method call.

Parameters:

`seconds` - the seconds to add, may be negative

Returns:

a `ZonedDateTime` based on this date-time with the seconds added, not null

Throws:

`DateTimeException` - if the result exceeds the supported date range

plusNanos

```
public ZonedDateTime plusNanos(long nanos)
```

Returns a copy of this `ZonedDateTime` with the specified number of nanoseconds added.

This operates on the instant time-line, such that adding one nano will always be a duration of one nano later. This may cause the local date-time to change by an amount other than one nano. Note that this is a different approach to that used by days, months and years.

This instance is immutable and unaffected by this method call.

Parameters:

`nanos` - the nanos to add, may be negative

Returns:

a `ZonedDateTime` based on this date-time with the nanoseconds added, not null

Throws:

`DateTimeException` - if the result exceeds the supported date range

minus

```
public ZonedDateTime minus(TemporalAmount amountToSubtract)
```

Returns a copy of this date-time with the specified amount subtracted.

This returns a `ZonedDateTime`, based on this one, with the specified amount subtracted. The amount is typically `Period` or `Duration` but may be any other type implementing the `TemporalAmount` interface.

The calculation is delegated to the amount object by calling `TemporalAmount.subtractFrom(Temporal)`. The amount implementation is free to implement the subtraction in any way it wishes, however it typically calls back to `minus(long, TemporalUnit)`. Consult the documentation of the amount implementation to determine if it can be successfully subtracted.

This instance is immutable and unaffected by this method call.

Specified by:

`minus` in interface `ChronoZonedDateTime<LocalDate>`

Specified by:

`minus` in interface `Temporal`

Parameters:

`amountToSubtract` - the amount to subtract, not null

Returns:

a `ZonedDateTime` based on this date-time with the subtraction made, not null

Throws:

`DateTimeException` - if the subtraction cannot be made

`ArithmeticException` - if numeric overflow occurs

minus

```
public ZonedDateTime minus(long amountToSubtract,
                           TemporalUnit unit)
```

Returns a copy of this date-time with the specified amount subtracted.

This returns a `ZonedDateTime`, based on this one, with the amount in terms of the unit subtracted. If it is not possible to subtract the amount, because the unit is not supported or for some other reason, an exception is thrown.

The calculation for date and time units differ.

Date units operate on the local time-line. The period is first subtracted from the local date-time, then converted back to a zoned date-time using the zone ID. The conversion uses `ofLocal(LocalDateTime, ZoneId, ZoneOffset)` with the offset before the subtraction.

Time units operate on the instant time-line. The period is first subtracted from the local date-time, then converted back to a zoned date-time using the zone ID. The conversion uses `ofInstant(LocalDateTime, ZoneOffset, ZoneId)` with the offset before the subtraction.

This method is equivalent to `plus(long, TemporalUnit)` with the amount negated. See that method for a full description of how addition, and thus subtraction, works.

This instance is immutable and unaffected by this method call.

Specified by:

`minus` in interface `ChronoZonedDateTime<LocalDate>`

Specified by:

`minus` in interface `Temporal`

Parameters:

`amountToSubtract` - the amount of the unit to subtract from the result, may be negative

`unit` - the unit of the amount to subtract, not null

Returns:

a `ZonedDateTime` based on this date-time with the specified amount subtracted, not null

Throws:

`DateTimeException` - if the subtraction cannot be made

`UnsupportedTemporalTypeException` - if the unit is not supported

`ArithmeticException` - if numeric overflow occurs

minusYears

`public ZonedDateTime minusYears(long years)`

Returns a copy of this `ZonedDateTime` with the specified number of years subtracted.

This operates on the local time-line, `subtracting years` to the local date-time. This is then converted back to a `ZonedDateTime`, using the zone ID to obtain the offset.

When converting back to `ZonedDateTime`, if the local date-time is in an overlap, then the offset will be retained if possible, otherwise the earlier offset will be used. If in a gap, the local date-time will be adjusted forward by the length of the gap.

This instance is immutable and unaffected by this method call.

Parameters:

`years` - the years to subtract, may be negative

Returns:

a `ZonedDateTime` based on this date-time with the years subtracted, not null

Throws:

`DateTimeException` - if the result exceeds the supported date range

minusMonths

`public ZonedDateTime minusMonths(long months)`

Returns a copy of this `ZonedDateTime` with the specified number of months subtracted.

This operates on the local time-line, `subtracting months` to the local date-time. This is then converted back to a `ZonedDateTime`, using the zone ID to obtain the offset.

When converting back to `ZonedDateTime`, if the local date-time is in an overlap, then the offset will be retained if possible, otherwise the earlier offset will be used. If in a gap, the local date-time will be adjusted forward by the length of the gap.

This instance is immutable and unaffected by this method call.

Parameters:

`months` - the months to subtract, may be negative

Returns:

a `ZonedDateTime` based on this date-time with the months subtracted, not null

Throws:

`DateTimeException` - if the result exceeds the supported date range

minusWeeks

```
public ZonedDateTime minusWeeks(long weeks)
```

Returns a copy of this `ZonedDateTime` with the specified number of weeks subtracted.

This operates on the local time-line, `subtracting weeks` to the local date-time. This is then converted back to a `ZonedDateTime`, using the zone ID to obtain the offset.

When converting back to `ZonedDateTime`, if the local date-time is in an overlap, then the offset will be retained if possible, otherwise the earlier offset will be used. If in a gap, the local date-time will be adjusted forward by the length of the gap.

This instance is immutable and unaffected by this method call.

Parameters:

`weeks` - the weeks to subtract, may be negative

Returns:

a `ZonedDateTime` based on this date-time with the weeks subtracted, not null

Throws:

`DateTimeException` - if the result exceeds the supported date range

minusDays

`public ZonedDateTime minusDays(long days)`

Returns a copy of this `ZonedDateTime` with the specified number of days subtracted.

This operates on the local time-line, `subtracting days` to the local date-time. This is then converted back to a `ZonedDateTime`, using the zone ID to obtain the offset.

When converting back to `ZonedDateTime`, if the local date-time is in an overlap, then the offset will be retained if possible, otherwise the earlier offset will be used. If in a gap, the local date-time will be adjusted forward by the length of the gap.

This instance is immutable and unaffected by this method call.

Parameters:

`days` - the days to subtract, may be negative

Returns:

a `ZonedDateTime` based on this date-time with the days subtracted, not null

Throws:

`DateTimeException` - if the result exceeds the supported date range

minusHours

`public ZonedDateTime minusHours(long hours)`

Returns a copy of this `ZonedDateTime` with the specified number of hours subtracted.

This operates on the instant time-line, such that subtracting one hour will always be a duration of one hour earlier. This may cause the local date-time to change by an amount other than one hour. Note that this is a different approach to that used by days, months and years, thus subtracting one day is not the same as adding 24 hours.

For example, consider a time-zone, such as 'Europe/Paris', where the Autumn DST cutover means that the local times 02:00 to 02:59 occur twice changing from offset +02:00 in summer to +01:00 in winter.

- Subtracting one hour from 03:30+01:00 will result in 02:30+01:00 (both in winter time)
- Subtracting one hour from 02:30+01:00 will result in 02:30+02:00 (moving from winter to summer time)

- Subtracting one hour from 02:30+02:00 will result in 01:30+02:00 (both in summer time)
- Subtracting three hours from 03:30+01:00 will result in 01:30+02:00 (moving from winter to summer time)

This instance is immutable and unaffected by this method call.

Parameters:

`hours` - the hours to subtract, may be negative

Returns:

a `ZonedDateTime` based on this date-time with the hours subtracted, not null

Throws:

`DateTimeException` - if the result exceeds the supported date range

minusMinutes

```
public ZonedDateTime minusMinutes(long minutes)
```

Returns a copy of this `ZonedDateTime` with the specified number of minutes subtracted.

This operates on the instant time-line, such that subtracting one minute will always be a duration of one minute earlier. This may cause the local date-time to change by an amount other than one minute. Note that this is a different approach to that used by days, months and years.

This instance is immutable and unaffected by this method call.

Parameters:

`minutes` - the minutes to subtract, may be negative

Returns:

a `ZonedDateTime` based on this date-time with the minutes subtracted, not null

Throws:

`DateTimeException` - if the result exceeds the supported date range

minusSeconds

```
public ZonedDateTime minusSeconds(long seconds)
```

Returns a copy of this `ZonedDateTime` with the specified number of seconds subtracted.

This operates on the instant time-line, such that subtracting one second will always be a duration of one second earlier. This may cause the local date-time

to change by an amount other than one second. Note that this is a different approach to that used by days, months and years.

This instance is immutable and unaffected by this method call.

Parameters:

`seconds` - the seconds to subtract, may be negative

Returns:

a `ZonedDateTime` based on this date-time with the seconds subtracted, not null

Throws:

`DateTimeException` - if the result exceeds the supported date range

minusNanos

```
public ZonedDateTime minusNanos(long nanos)
```

Returns a copy of this `ZonedDateTime` with the specified number of nanoseconds subtracted.

This operates on the instant time-line, such that subtracting one nano will always be a duration of one nano earlier. This may cause the local date-time to change by an amount other than one nano. Note that this is a different approach to that used by days, months and years.

This instance is immutable and unaffected by this method call.

Parameters:

`nanos` - the nanos to subtract, may be negative

Returns:

a `ZonedDateTime` based on this date-time with the nanoseconds subtracted, not null

Throws:

`DateTimeException` - if the result exceeds the supported date range

query

```
public <R> R query(TemporalQuery<R> query)
```

Queries this date-time using the specified query.

This queries this date-time using the specified query strategy object. The `TemporalQuery` object defines the logic to be used to obtain the result. Read the documentation of the query to understand what the result of this method will be.

The result of this method is obtained by invoking the `TemporalQuery.queryFrom(TemporalAccessor)` method on the specified query passing `this` as the argument.

Specified by:

`query` in interface `ChronoZonedDateTime<LocalDate>`

Specified by:

`query` in interface `TemporalAccessor`

Type Parameters:

`R` - the type of the result

Parameters:

`query` - the query to invoke, not null

Returns:

the query result, null may be returned (defined by the query)

Throws:

`DateTimeException` - if unable to query (defined by the query)

`ArithmeticException` - if numeric overflow occurs (defined by the query)

until

```
public long until(Temporal endExclusive,
                  TemporalUnit unit)
```

Calculates the amount of time until another date-time in terms of the specified unit.

This calculates the amount of time between two `ZonedDateTime` objects in terms of a single `TemporalUnit`. The start and end points are `this` and the specified date-time. The result will be negative if the end is before the start. For example, the amount in days between two date-times can be calculated using `startDateTime.until(endDateTime, DAYS)`.

The `Temporal` passed to this method is converted to a `ZonedDateTime` using `from(TemporalAccessor)`. If the time-zone differs between the two zoned date-times, the specified end date-time is normalized to have the same zone as this date-time.

The calculation returns a whole number, representing the number of complete units between the two date-times. For example, the amount in months between 2012-06-15T00:00Z and 2012-08-14T23:59Z will only be one month as it is one minute short of two months.

There are two equivalent ways of using this method. The first is to invoke this method. The second is to use `TemporalUnit.between(Temporal, Temporal)`:

```
// these two lines are equivalent
amount = start.until(end, MONTHS);
amount = MONTHS.between(start, end);
```

The choice should be made based on which makes the code more readable.

The calculation is implemented in this method for `ChronoUnit`. The units `NANOS`, `MICROS`, `MILLIS`, `SECONDS`, `MINUTES`, `HOURS` and `HALF_DAYS`, `DAYS`, `WEEKS`, `MONTHS`, `YEARS`, `DECADES`, `CENTURIES`, `MILLENNIA` and `ERAS` are supported. Other `ChronoUnit` values will throw an exception.

The calculation for date and time units differ.

Date units operate on the local time-line, using the local date-time. For example, the period from noon on day 1 to noon the following day in days will always be counted as exactly one day, irrespective of whether there was a daylight savings change or not.

Time units operate on the instant time-line. The calculation effectively converts both zoned date-times to instants and then calculates the period between the instants. For example, the period from noon on day 1 to noon the following day in hours may be 23, 24 or 25 hours (or some other amount) depending on whether there was a daylight savings change or not.

If the unit is not a `ChronoUnit`, then the result of this method is obtained by invoking `TemporalUnit.between(Temporal, Temporal)` passing `this` as the first argument and the converted input temporal as the second argument.

This instance is immutable and unaffected by this method call.

Specified by:

`until` in interface `Temporal`

Parameters:

`endExclusive` - the end date, exclusive, which is converted to a `ZonedDateTime`, not null

`unit` - the unit to measure the amount in, not null

Returns:

the amount of time between this date-time and the end date-time

Throws:

`DateTimeException` - if the amount cannot be calculated, or the end temporal cannot be converted to a `ZonedDateTime`

`UnsupportedTemporalTypeException` - if the unit is not supported

`ArithmeticException` - if numeric overflow occurs

format

`public String format(DateTimeFormatter formatter)`

Formats this date-time using the specified formatter.

This date-time will be passed to the formatter to produce a string.

Specified by:

`format` in interface `ChronoZonedDateTime<LocalDate>`

Parameters:

`formatter` - the formatter to use, not null

Returns:

the formatted date-time string, not null

Throws:

`DateTimeException` - if an error occurs during printing

toOffsetDateTime

`public OffsetDateTime toOffsetDateTime()`

Converts this date-time to an `OffsetDateTime`.

This creates an offset date-time using the local date-time and offset. The zone ID is ignored.

Returns:

an offset date-time representing the same local date-time and offset, not null

equals

`public boolean equals(Object obj)`

Checks if this date-time is equal to another date-time.

The comparison is based on the offset date-time and the zone. Only objects of type `ZonedDateTime` are compared, other types return false.

Specified by:

`equals` in interface `ChronoZonedDateTime<LocalDate>`

Overrides:

`equals` in class `Object`

Parameters:

`obj` - the object to check, null returns false

Returns:

true if this is equal to the other date-time

See Also:

`Object.hashCode()`, `HashMap`

hashCode

```
public int hashCode()
```

A hash code for this date-time.

Specified by:

`hashCode` in interface `ChronoZonedDateTime<LocalDate>`

Overrides:

`hashCode` in class `Object`

Returns:

a suitable hash code

See Also:

`Object.equals(java.lang.Object)`,
`System.identityHashCode(Object)`

toString

```
public String toString()
```

Outputs this date-time as a `String`, such as `2007-12-03T10:15:30+01:00[Europe/Paris]`.

The format consists of the output of `LocalDateTime.toString()`, followed by the output of `ZoneOffset.toString()`. If the time has zero seconds and/or nanoseconds, they are omitted to produce the shortest representation. If the `ZoneId` is not the same as the offset, then the ID is output. The output is compatible with ISO-8601 if the offset and ID are the same, and the seconds in the offset are zero.

Specified by:

`toString` in interface `ChronoZonedDateTime<LocalDate>`

Overrides:

`toString` in class `Object`

Returns:

a string representation of this date-time, not null

Report a bug or suggest an enhancement

For further API reference and developer documentation see the Java SE Documentation, which contains more detailed, developer-targeted descriptions with conceptual overviews, definitions of terms, workarounds, and working code examples. Other versions.

Java is a trademark or registered trademark of Oracle and/or its affiliates in the US and other countries.

Copyright © 1993, 2026, Oracle and/or its affiliates, 500 Oracle Parkway, Redwood Shores, CA 94065 USA.

All rights reserved. Use is subject to license terms and the documentation redistribution policy.

Scripting on this page tracks web page traffic, but does not change the content in any way.